



01000101 01110011 01100011 01101111 01101100 01100001 00100000
01100100 01100101 00100000 01000100 01100001 01100100
01101111 01110011 00001010

GLOSSÁRIO SQL

SQLITE ONLINE: EXECUTANDO
CONSULTAS SQL

BOAS-VINDAS AO GLOSSÁRIO DE SQL!

Este glossário foi criado para proporcionar uma introdução clara e concisa a alguns comandos e funções do SQL, como as cláusulas **LIMIT, LIKE, GROUP BY, HAVING**, funções de agregação, funções específicas para trabalhar com cada tipo de dado, etc. Explore e aprofunde-se no mundo dos bancos de dados, utilizando este guia para fortalecer sua compreensão e aplicação do SQL. Aproveite o material e, em caso de dúvidas, sinta-se à vontade para enviar perguntas através do fórum do curso.

Um abraço e bons estudos!

SUMÁRIO

01000101 0110011 01000101 01101111 01101001 01000001
00100000 01101001 01000101 01000000 01000100 01100001
01101001 01101111 0110011 00001001

✿ UTILIZANDO A CLÁUSULA LIMIT.....	04
✿ OPERADORES IS NULL E NOT NULL.....	07
✿ UTILIZANDO O COMANDO LIKE	10
✿ OPERADORES LÓGICOS.....	13
✿ FUNÇÕES DE AGREGAÇÃO.....	18
✿ CLÁUSULA GROUP BY.....	25
✿ CLÁUSULA HAVING.....	28

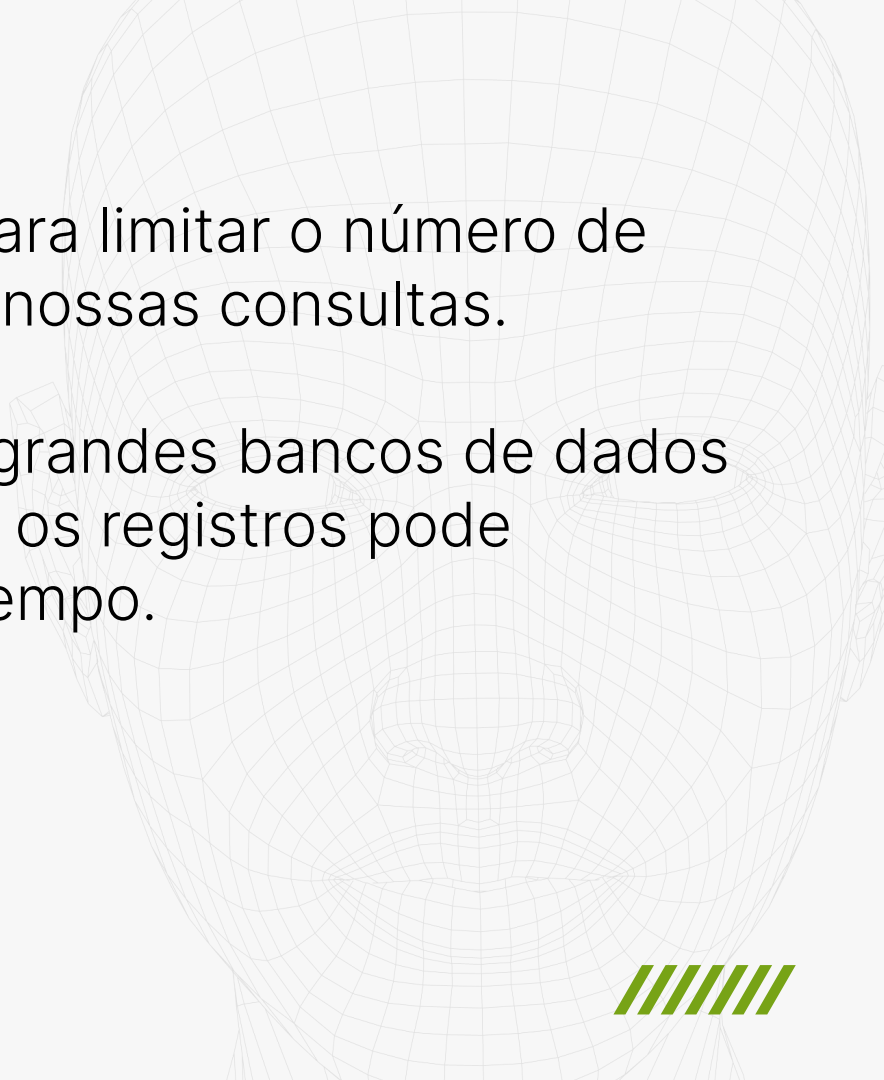
✿ FUNÇÕES DE STRING.....	31
✿ FUNÇÕES DE DATA.....	36
✿ FUNÇÕES NUMÉRICAS.....	41
✿ FUNÇÕES DE CONVERSÃO	46
✿ CASE WHEN	50
✿ CLÁUSULA RENAME	55

01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100011
00001010

// Glossário SQL_

UTILIZANDO A CLÁUSULA LIMIT



A faint, stylized wireframe of a human face is visible in the background, composed of a grid of lines forming the facial structure.

A cláusula **LIMIT** é utilizada para limitar o número de registros que é retornado em nossas consultas.

Ela é particularmente útil em grandes bancos de dados onde a recuperação de todos os registros pode consumir muitos recursos e tempo.



```
SELECT * FROM HistoricoEmprego  
ORDER BY salario DESC  
LIMIT 5;
```

Com essa consulta buscamos os dados da tabela **HistoricoEmprego** ordenando por salário de maneira decrescente e limitamos o resultado da consulta em apenas 5 registros com a cláusula **LIMIT**.

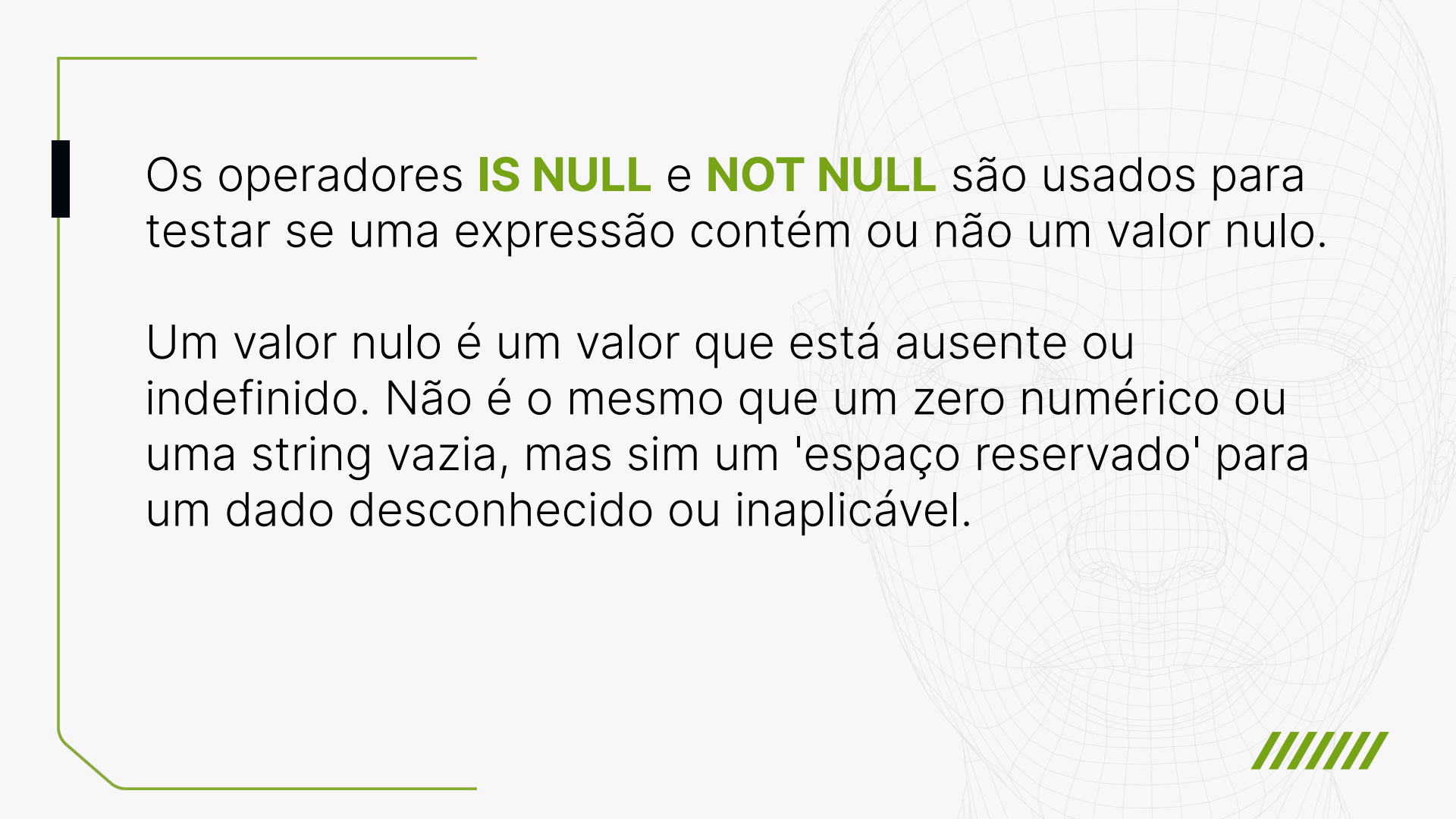


01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100111
00001010

// Glossário SQL_

OPERADORES IS NULL E NOT NULL





Os operadores **IS NULL** e **NOT NULL** são usados para testar se uma expressão contém ou não um valor nulo.

Um valor nulo é um valor que está ausente ou indefinido. Não é o mesmo que um zero numérico ou uma string vazia, mas sim um 'espaço reservado' para um dado desconhecido ou inaplicável.



Por exemplo, se quisermos filtrar na tabela de **HistoricoEmprego** apenas os registros que possuam um valor nulo na coluna **datatermino**:

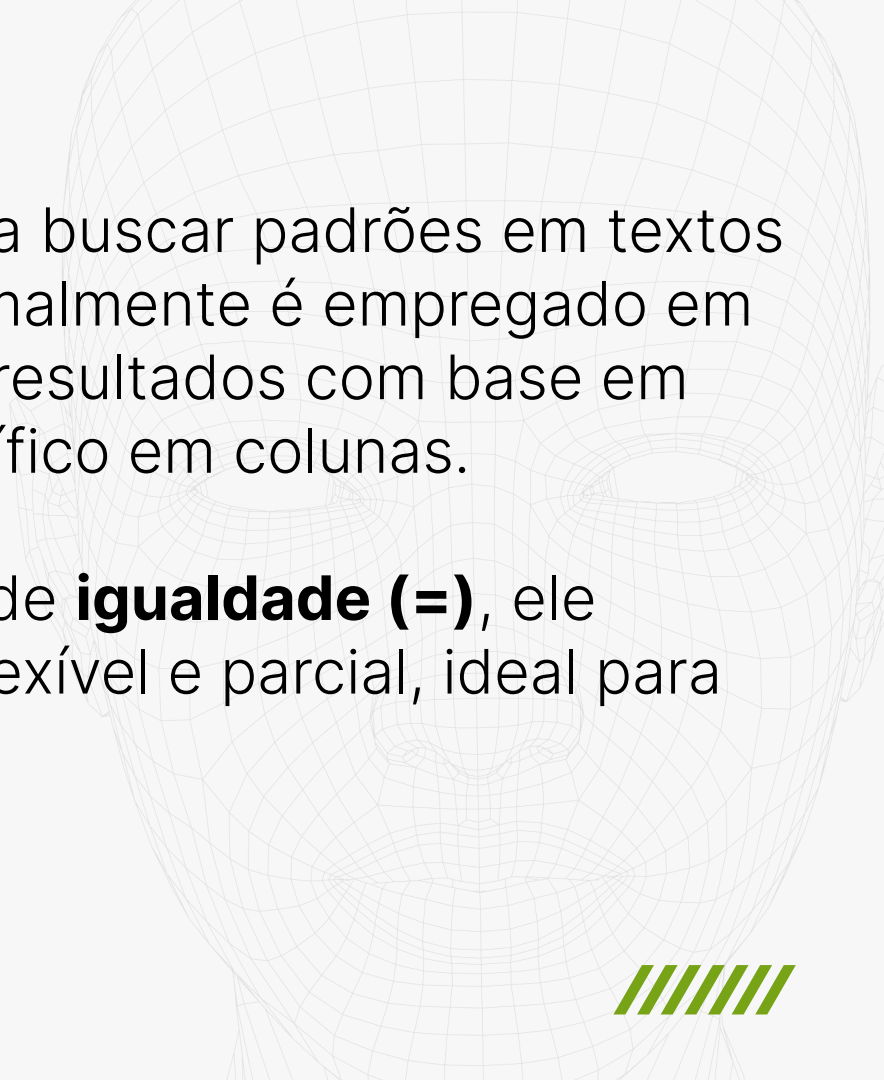
```
SELECT * FROM HistoricoEmprego  
WHERE datatermino IS NULL  
ORDER BY salario DESC  
LIMIT 5;  
  
;
```

01000101 0110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 0110011
00001010

// Glossário SQL_

UTILIZANDO O COMANDO LIKE



A faint, stylized wireframe of a human face is visible in the background, composed of a grid of lines forming the facial structure.

O comando **LIKE** é usado para buscar padrões em textos dentro de uma consulta. Normalmente é empregado em cláusulas **WHERE** para filtrar resultados com base em algum padrão de texto específico em colunas.

Diferentemente do operador de **igualdade (=)**, ele permite uma pesquisa mais flexível e parcial, ideal para pesquisas em texto livre.



Nessa consulta temos um exemplo, buscando campos que possuam a palavra realizar:

```
SELECT * FROM Treinamento  
WHERE curso LIKE '%realizar%';
```

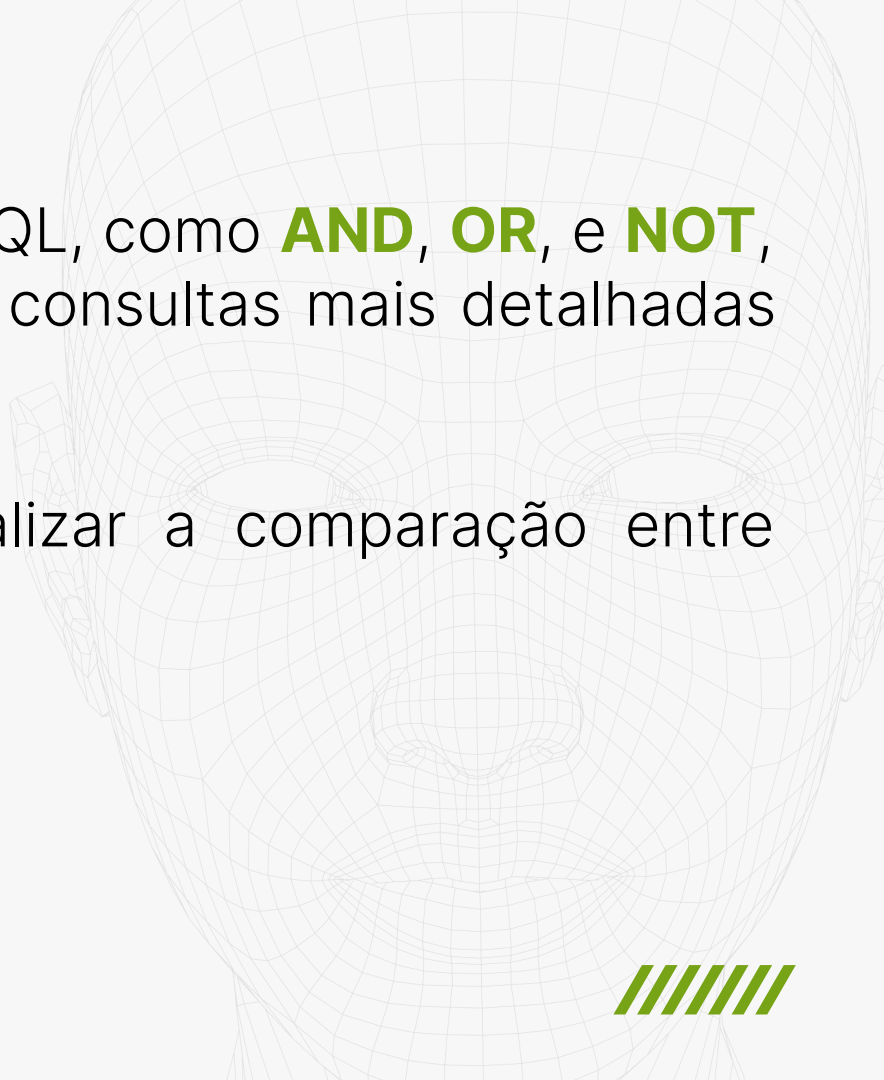


01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100111
00001010

// Glossário SQL_

OPERADORES LÓGICOS

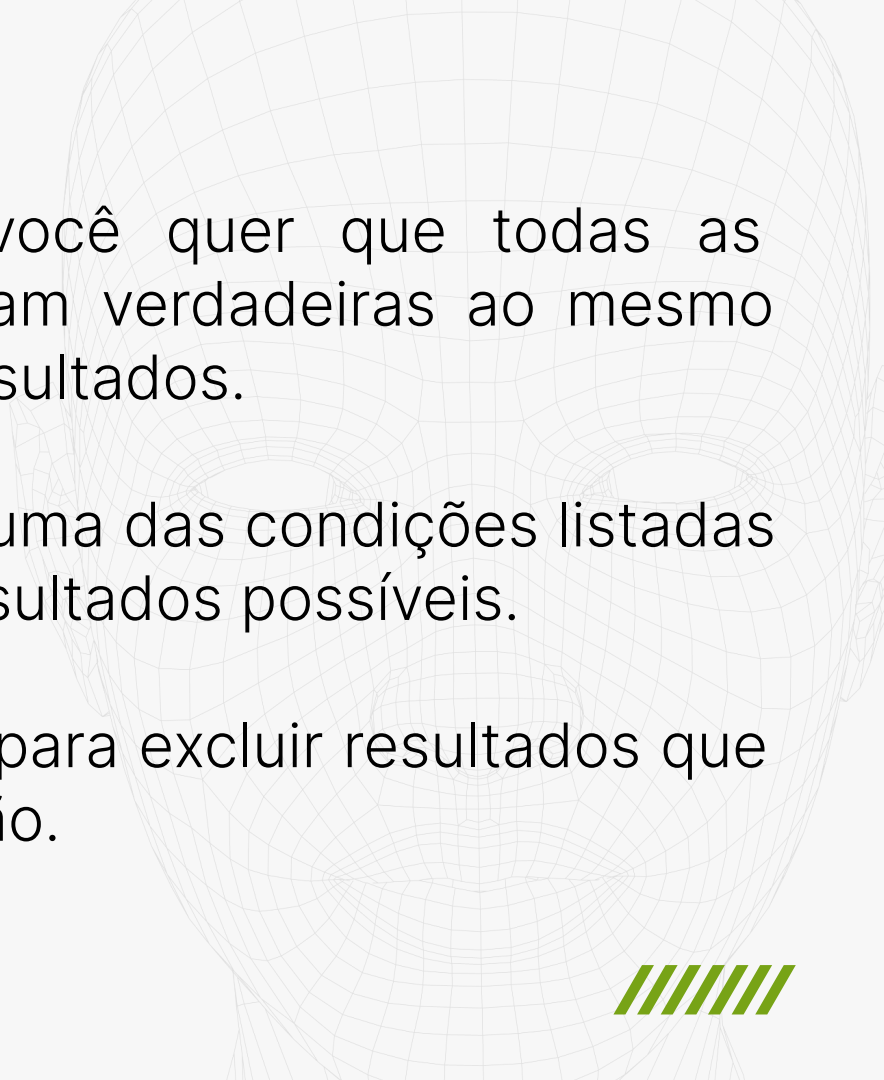


A faint, light gray wireframe of a human face is visible in the background on the right side of the slide.

Os **operadores lógicos** em SQL, como **AND**, **OR**, e **NOT**, são essenciais para elaborar consultas mais detalhadas e precisas.

Eles são utilizados para realizar a comparação entre expressões.





O **AND** é usado quando você quer que todas as condições especificadas sejam verdadeiras ao mesmo tempo, ideal para afinar os resultados.

O **OR** é útil quando qualquer uma das condições listadas serve, ampliando assim os resultados possíveis.

Já o **NOT** é o inverso, usado para excluir resultados que atendem a uma certa condição.



Por exemplo, para filtrar as pessoas colaboradoras que possuam como cargo **Oftalmologista** ou **Dermatologista**, usamos a seguinte consulta:

```
SELECT * FROM HistoricoEmprego  
WHERE cargo = 'Oftalmologista' OR  
cargo = 'Dermatologista';
```



Já nessa consulta excluimos do resultado todas as pessoas colaboradoras que possuam como cargo **Oftalmologista, Dermatologista e Professor**:

```
SELECT * FROM HistoricoEmprego  
WHERE cargo NOT IN ('Oftalmologista',  
'Dermatologista', 'Professor');
```

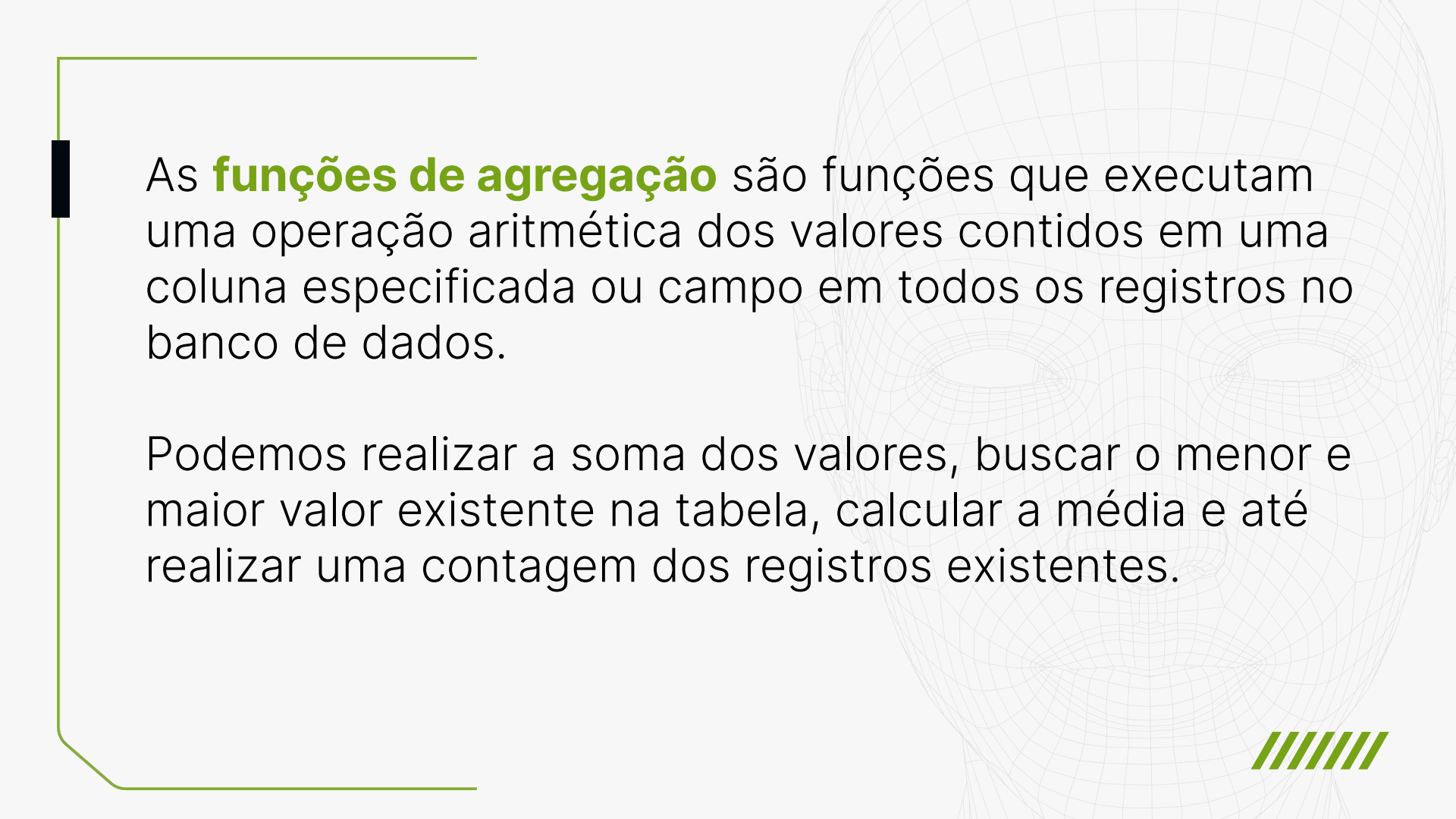


01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100111
00001010

// Glossário SQL_

FUNÇÕES DE AGREGAÇÃO



A faint, light gray wireframe of a human face is visible in the background, centered and slightly to the right. It consists of a grid of lines forming the facial structure.

As **funções de agregação** são funções que executam uma operação aritmética dos valores contidos em uma coluna especificada ou campo em todos os registros no banco de dados.

Podemos realizar a soma dos valores, buscar o menor e maior valor existente na tabela, calcular a média e até realizar uma contagem dos registros existentes.



Por exemplo, para realizar a soma dos valores de um campo, utilizamos a função de agregação **SUM**:

```
SELECT Mês, SUM(Faturamento) FROM  
TabelaFaturamento;
```

Por exemplo, para buscar o valor máximo dos valores de um campo, utilizamos a função de agregação **MAX**:

```
SELECT Mês, MAX(Faturamento) FROM  
TabelaFaturamento;
```

Já, para buscar o valor mínimo dos valores de um campo, utilizamos a função de agregação **MIN**:

```
SELECT Mês, MIN(Faturamento) FROM  
TabelaFaturamento;
```

Para calcular a média dos valores de um campo, utilizamos a função de agregação **AVG**:

```
SELECT AVG(Faturamento) FROM  
TabelaFaturamento;
```


Para realizar a contagem dos registros existentes em uma tabela, utilizamos a função de agregação **COUNT**:

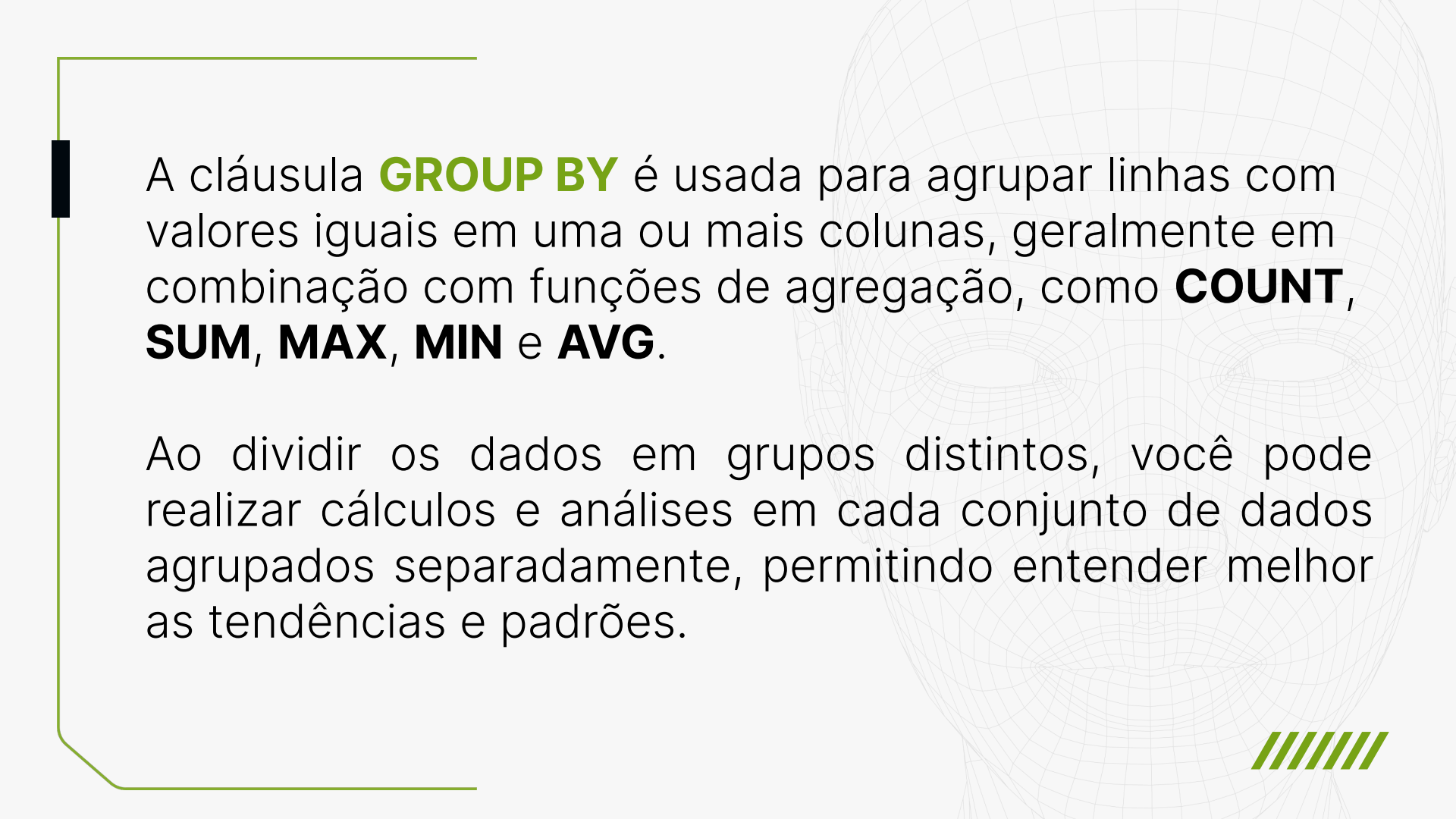
```
SELECT COUNT (DISTINCT cargo) FROM  
HistoricoEmprego;
```

01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100111
00001010

// Glossário SQL_

CLÁUSULA GROUP BY



A faint, light gray wireframe of a human face is visible in the background, centered and slightly to the right. It consists of a grid of lines forming the facial structure.

A cláusula **GROUP BY** é usada para agrupar linhas com valores iguais em uma ou mais colunas, geralmente em combinação com funções de agregação, como **COUNT**, **SUM**, **MAX**, **MIN** e **AVG**.

Ao dividir os dados em grupos distintos, você pode realizar cálculos e análises em cada conjunto de dados agrupados separadamente, permitindo entender melhor as tendências e padrões.



Por exemplo, nessa consulta trazemos os registros de **parentesco** da tabela **Dependentes** agrupados pelo tipo de parentesco.

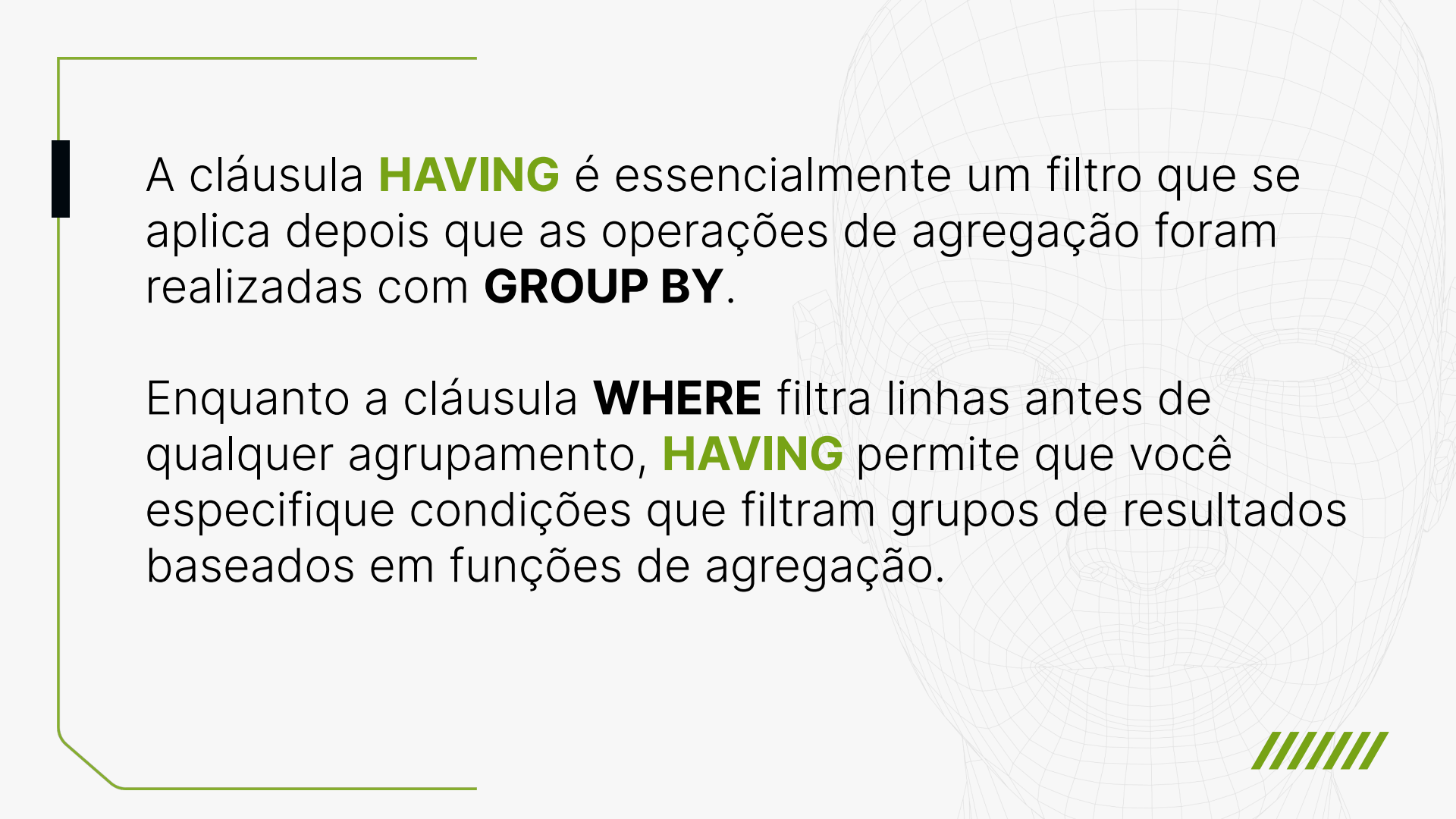
```
SELECT parentesco, COUNT(*) FROM  
Dependentes  
GROUP BY parentesco;
```

01000101 0110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 0110011
00001010

// Glossário SQL_

CLÁUSULA HAVING





A cláusula **HAVING** é essencialmente um filtro que se aplica depois que as operações de agregação foram realizadas com **GROUP BY**.

Enquanto a cláusula **WHERE** filtra linhas antes de qualquer agrupamento, **HAVING** permite que você especifique condições que filtram grupos de resultados baseados em funções de agregação.



Por exemplo, nessa consulta que possui um agrupamento por cargos, criamos uma condição que traga apenas os registros de cargos que se repitam, ou seja, sejam contados duas ou mais vezes na tabela.

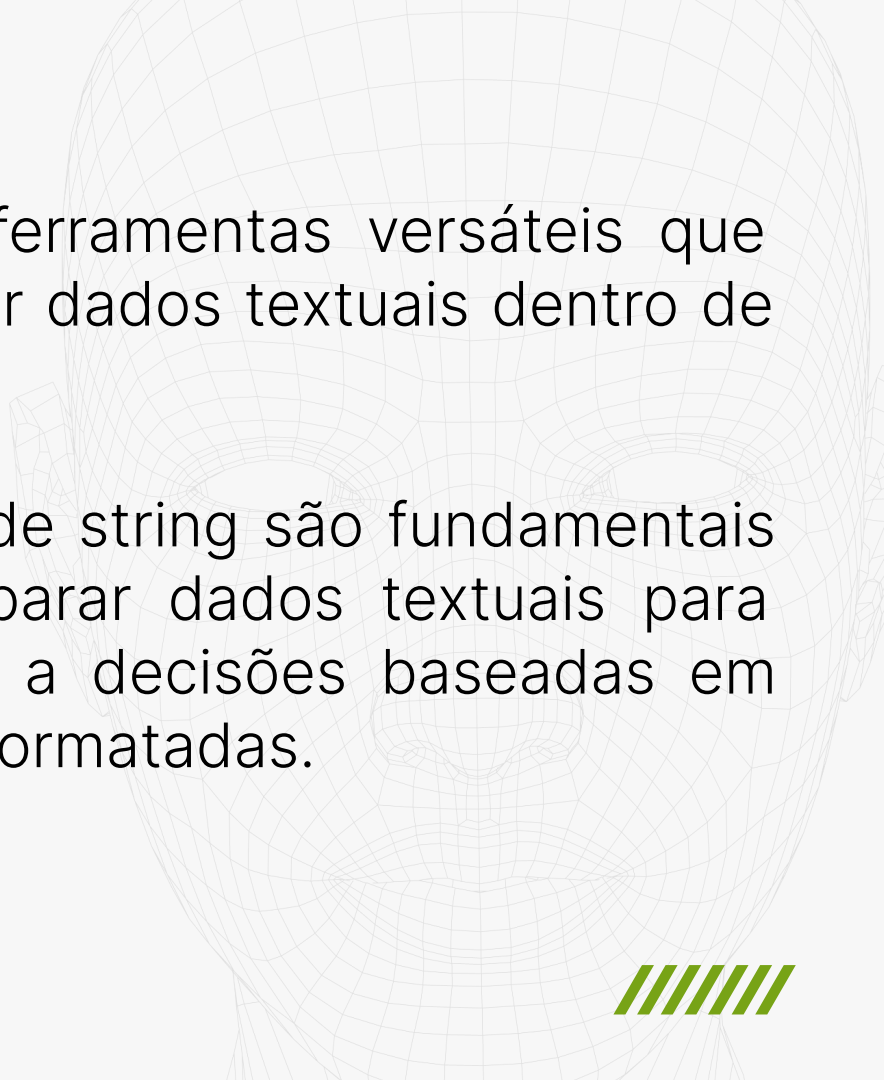
```
SELECT cargo, COUNT(*) qtd  
FROM HistoricoEmprego  
GROUP BY cargo  
HAVING qtd >=2;
```

01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100111
00001010

// Glossário SQL_

FUNÇÕES DE STRING

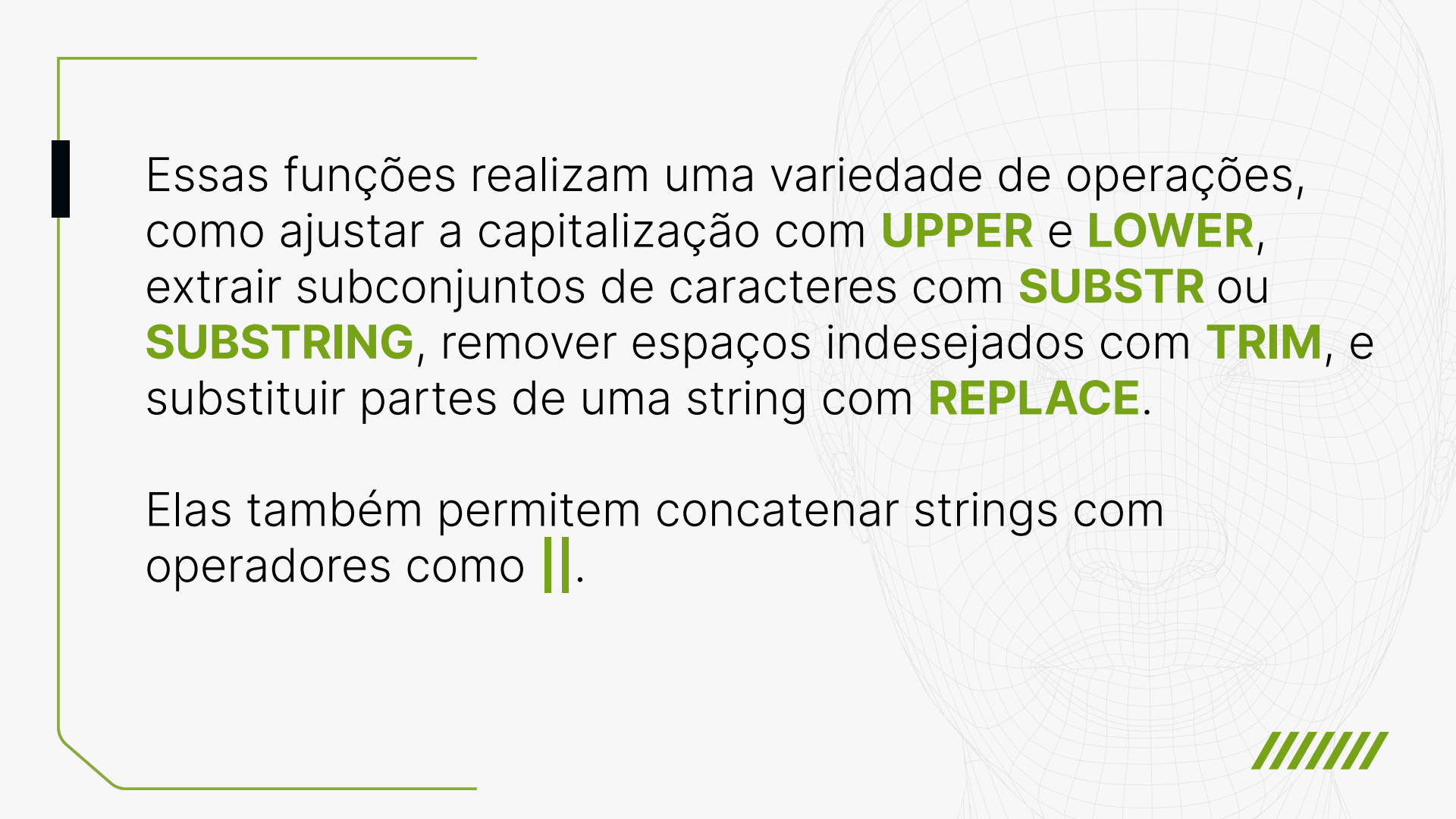


A faint, stylized wireframe of a human face is visible in the background, composed of a grid of lines forming the facial structure.

As **funções de string** são ferramentas versáteis que permitem manipular e analisar dados textuais dentro de um banco de dados.

Essencialmente, as funções de string são fundamentais para limpar, formatar e preparar dados textuais para análise, relatórios e suporte a decisões baseadas em informações precisas e bem formatadas.





Essas funções realizam uma variedade de operações, como ajustar a capitalização com **UPPER** e **LOWER**, extrair subconjuntos de caracteres com **SUBSTR** ou **SUBSTRING**, remover espaços indesejados com **TRIM**, e substituir partes de uma string com **REPLACE**.

Elas também permitem concatenar strings com operadores como **||**.



Por exemplo, nessa consulta utilizamos a função **LENGTH** para conferir se todas as linhas de registros da coluna **CPF** da tabela **Colaboradores** contenham 11 caracteres, ou seja, tenham um número de CPF preenchido sem faltar nenhum dígito:

```
SELECT COUNT(*), LENGTH(cpf) qtd  
FROM Colaboradores  
WHERE qtd = 11;
```

Já nessa outra consulta, criamos uma frase usando o operador **||** para concatenar textos com informações contidas em algumas colunas da tabela Colaboradores. Além de aplicar a função **UPPER** para trazer todo o texto com caracteres maiúsculos.

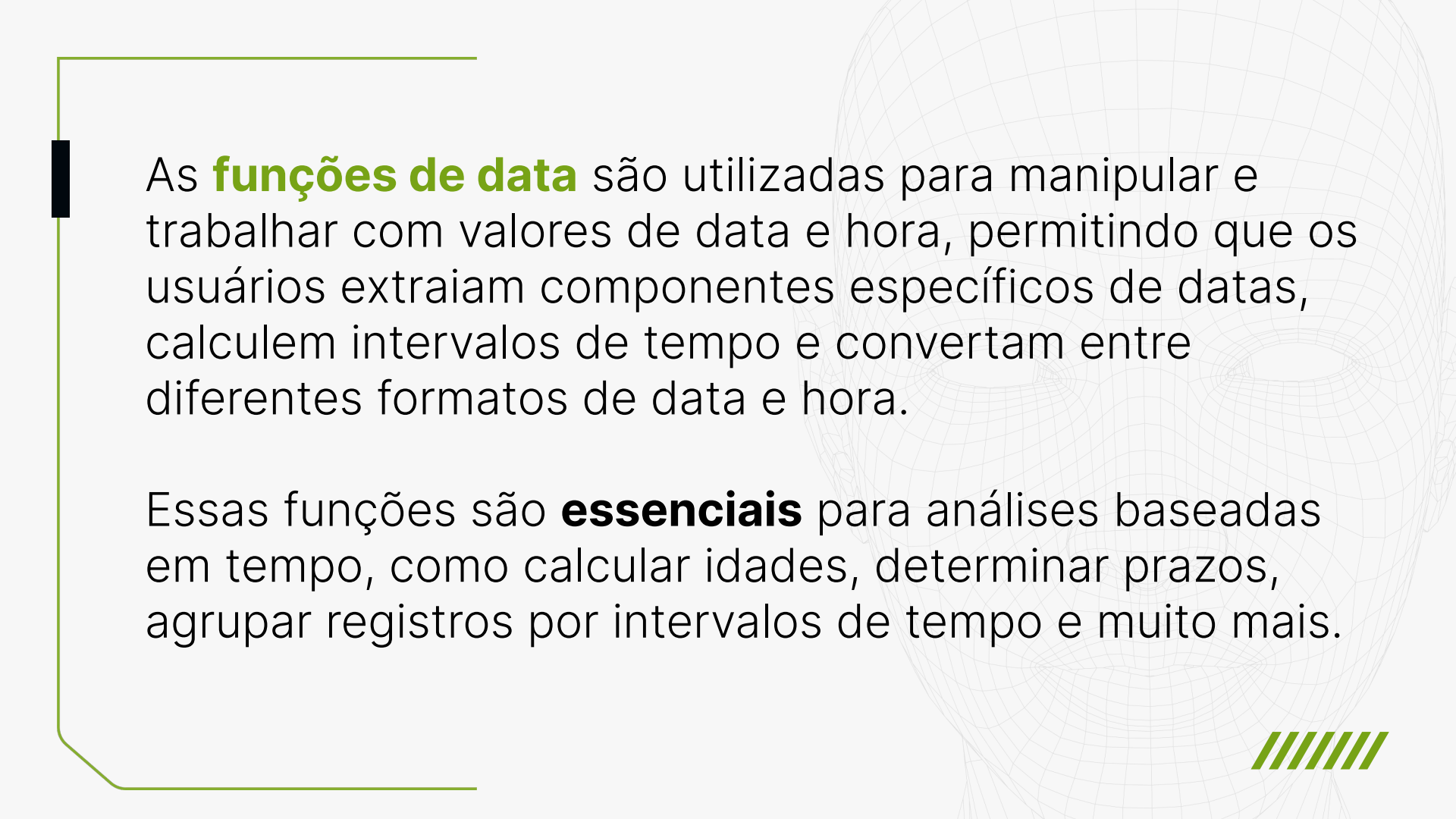
```
SELECT UPPER('A pessoa colaboradora ' || nome || ' de CPF ' || cpf || ' possui o seguinte endereço: ' || endereco) as texto  
FROM Colaboradores;
```

01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100111
00001010

// Glossário SQL_

FUNÇÕES DE DATA





As **funções de data** são utilizadas para manipular e trabalhar com valores de data e hora, permitindo que os usuários extraiam componentes específicos de datas, calculem intervalos de tempo e convertam entre diferentes formatos de data e hora.

Essas funções são **essenciais** para análises baseadas em tempo, como calcular idades, determinar prazos, agrupar registros por intervalos de tempo e muito mais.



Funções como **DATE()**, **TIME()**, e **DATETIME()** são usadas para recuperar a data ou hora atual ou converter strings em valores de data/hora. **STRFTIME()** é uma função versátil para formatar datas e horas de acordo com padrões específicos.

Além disso, funções como **JULIANDAY()** podem calcular a diferença entre datas ou converter datas para um formato numérico que representa dias.



Por exemplo, nessa consulta usamos a função **STRFTIME** para trazer a coluna **datainicio** da tabela **Licencas** no formato ano/mês:

```
SELECT id_colaborador, STRFTIME( '%Y/%m' ,  
datainicio) FROM Licencas;
```



Já nessa consulta, usamos a função **JULIANDAY** para trazer o valor da diferença em dias entre as datas da coluna **datatermino** e **datacontratacao** da tabela **HistoricoEmprego**:

```
SELECT id_colaborador, JULIANDAY (datatermino) -  
JULIANDAY (datacontratacao)  
FROM HistoricoEmprego  
WHERE datatermino IS NOT NULL;
```

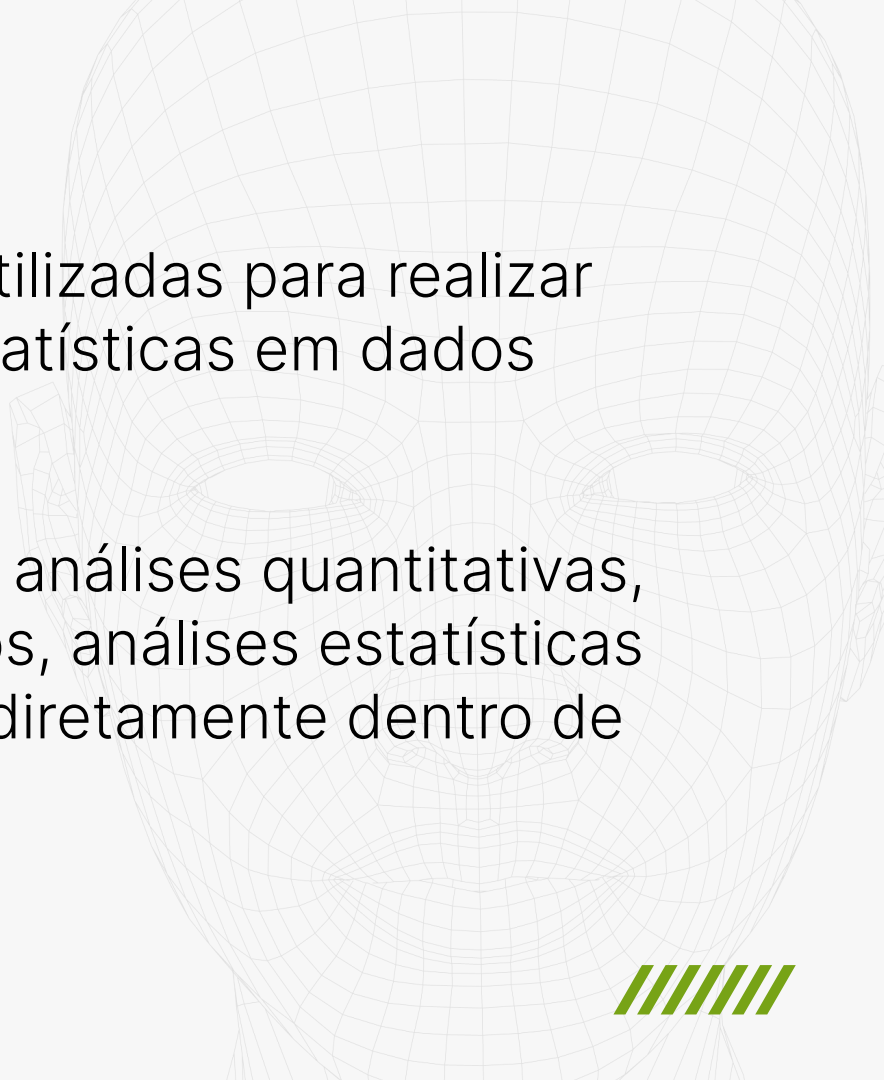


01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100111
00001010

// Glossário SQL_

FUNÇÕES NUMÉRICAS



A faint, light gray wireframe of a human face is visible in the background, centered on the right side of the slide. It consists of a grid of lines forming the facial structure.

As **funções numéricas** são utilizadas para realizar operações matemáticas e estatísticas em dados numéricos.

Essas funções são vitais para análises quantitativas, permitindo cálculos complexos, análises estatísticas e transformações numéricas diretamente dentro de consultas SQL.



Essas incluem funções como **ABS()** para obter o valor absoluto, **ROUND()** para arredondar números a uma certa precisão decimal, **CEIL()** e **FLOOR()** para arredondar números para o inteiro mais próximo acima ou abaixo, respectivamente. Além disso, **RANDOM()** gera números aleatórios.



Por exemplo, nessa consulta usamos a função **ROUND** para arredondar a média do faturamento bruto para um número com 2 casas decimais.

```
SELECT AVG(faturamento_bruto), ROUND  
(AVG(faturamento_bruto),2) FROM faturamento;
```

Já nessa outra consulta, usamos a função **FLOOR** para arredondar os valores das colunas **faturamento_bruto** e **despesas** para o número inteiro menor mais próximo:

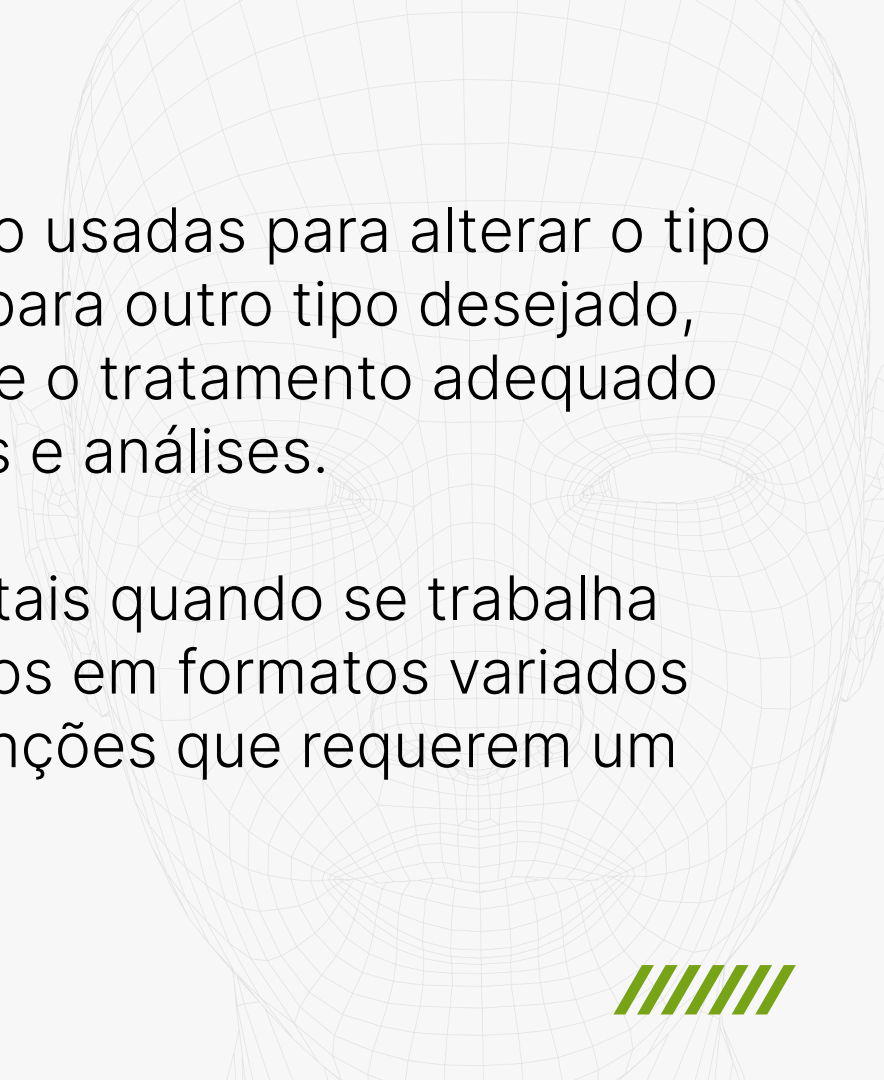
```
SELECT FLOOR(faturamento_bruto), FLOOR(despesas)  
FROM faturamento;
```

01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100011
00001010

// Glossário SQL_

FUNÇÕES DE CONVERSÃO



A faint, light green wireframe of a human face is visible in the background, centered on the right side of the slide.

As **funções de conversão** são usadas para alterar o tipo de dados de uma expressão para outro tipo desejado, facilitando a compatibilidade e o tratamento adequado dos dados durante operações e análises.

Essas funções são fundamentais quando se trabalha com dados que foram inseridos em formatos variados ou ao preparar dados para funções que requerem um tipo específico.



A função mais comum é **CAST()**, que converte explicitamente valores de um tipo para outro, como de texto para inteiro ou de data para texto. Outras funções específicas de sistemas, como **CONVERT()** no **SQL Server**, oferecem funcionalidades similares com sintaxes diferentes ou recursos adicionais.



Por exemplo, nessa consulta concatenamos um texto com a coluna de faturamento bruto, que possui dados do tipo numérico, então para que isso seja possível, aplicamos a função **CAST** e convertemos os dados numéricos em texto.

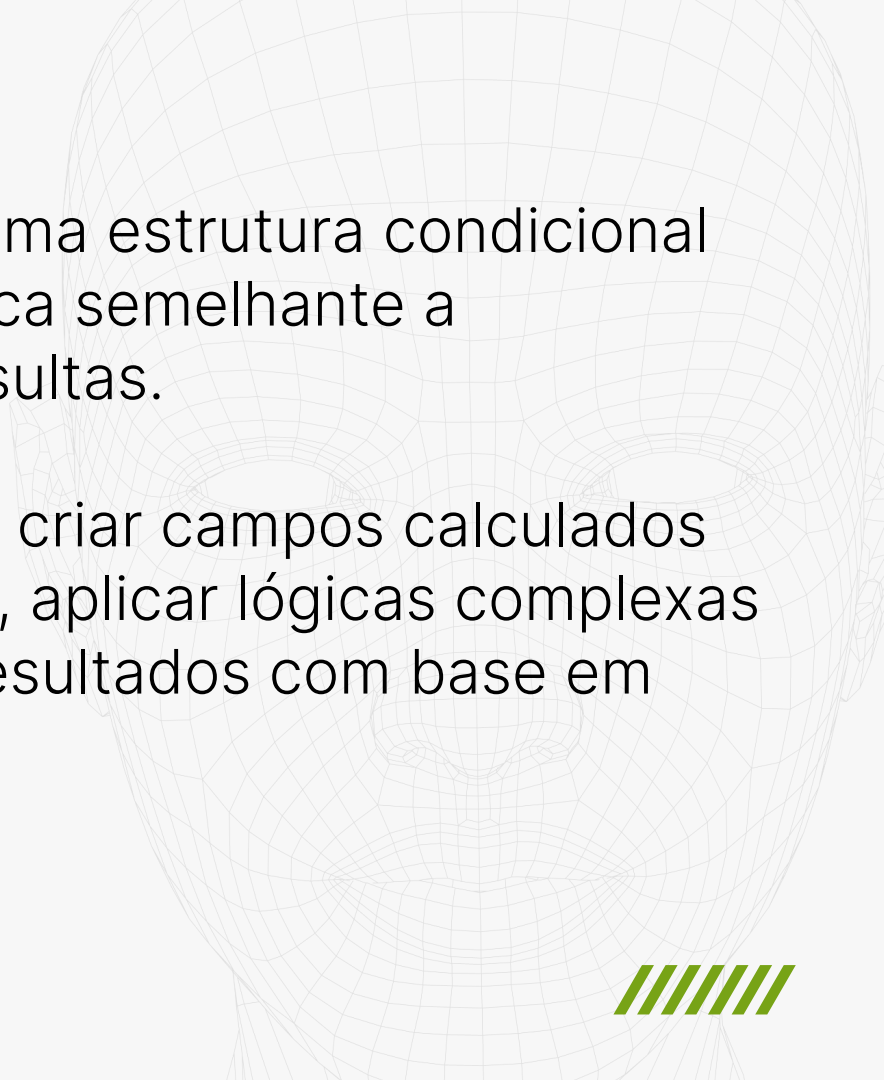
```
SELECT (' 0 faturamento bruto médio foi ' ||  
CAST(ROUND (AVG(faturamento_bruto),2) AS TEXT))  
FROM faturamento;
```

01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100111
00001010

// Glossário SQL_

CASE WHEN

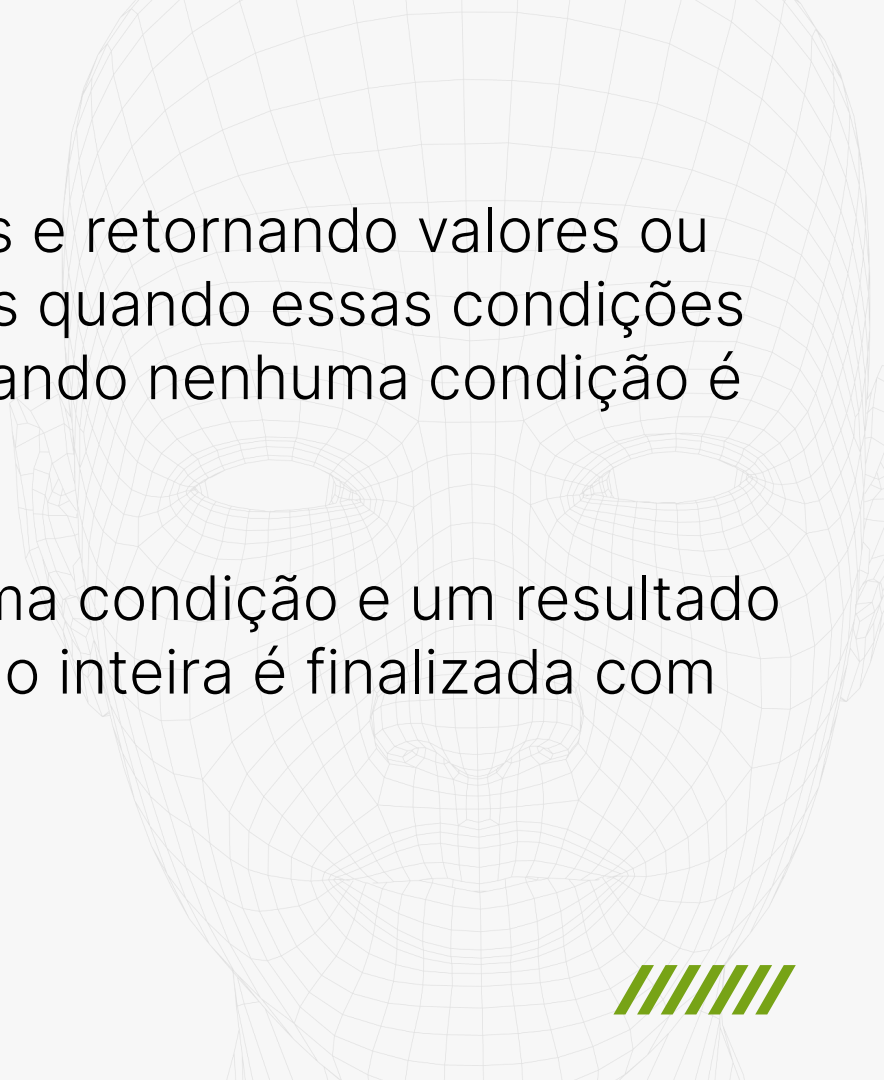


A faint, light gray wireframe of a human face is visible in the background on the right side of the slide.

A expressão **CASE WHEN** é uma estrutura condicional que permite implementar lógica semelhante a **"if-then-else"** dentro de consultas.

Isso é extremamente útil para criar campos calculados dinâmicos, categorizar dados, aplicar lógicas complexas em consultas e transformar resultados com base em critérios específicos.



A faint, stylized wireframe of a human face is visible in the background, composed of a grid of lines forming the facial structure.

Funciona avaliando condições e retornando valores ou executando ações específicas quando essas condições são atendidas (**WHEN**) ou quando nenhuma condição é satisfeita (**ELSE**).

Cada **WHEN** é seguido por uma condição e um resultado correspondente, e a expressão inteira é finalizada com **END**.



Por exemplo, podemos utilizar a cláusula **CASE**, para criar uma condição e categorizar os salários dos colaboradores em **baixo**, **médio** e **alto**.



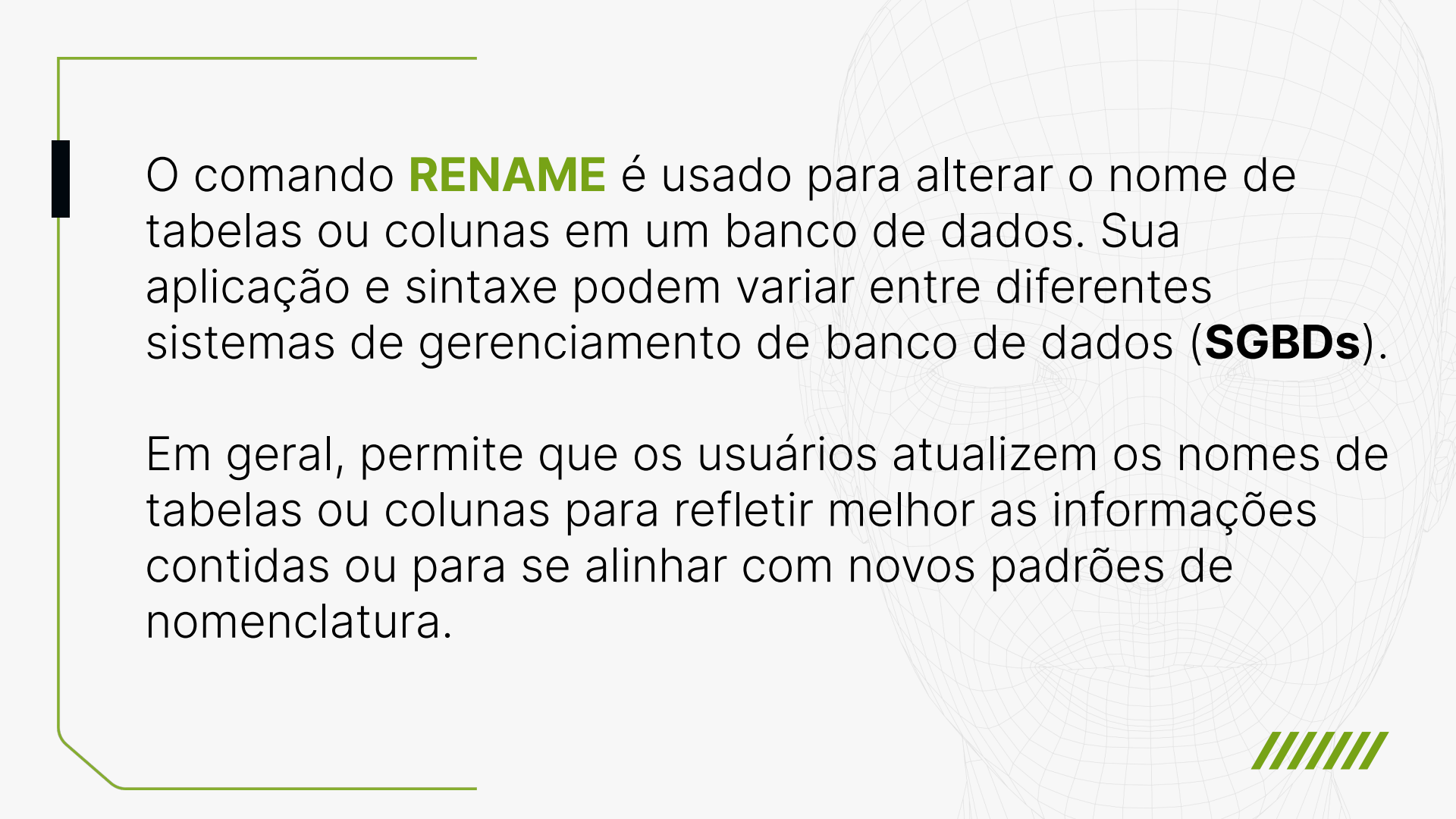
```
SELECT
    id,
    id_funcionario,
    cargo,
    salario,
CASE
    WHEN salario < 3000 THEN 'Baixo'
    WHEN salario BETWEEN 3000 AND 6000 THEN 'Médio'
    ELSE 'Alto'
END AS categoria_salario
FROM
    HistoricoEmprego;
```

01000101 01110011 01100011 01101111 01101100
01100001 00100000 01100100 01100101 00100000
01000100 01100001 01100100 01101111 01100111
00001010

// Glossário SQL_

CLÁUSULA RENAME



A decorative background featuring a faint wireframe of a human head. A solid green line runs vertically on the left side, with a horizontal segment at the top and bottom, forming a partial frame around the text.

O comando **RENAME** é usado para alterar o nome de tabelas ou colunas em um banco de dados. Sua aplicação e sintaxe podem variar entre diferentes sistemas de gerenciamento de banco de dados (**SGBDs**).

Em geral, permite que os usuários atualizem os nomes de tabelas ou colunas para refletir melhor as informações contidas ou para se alinhar com novos padrões de nomenclatura.



// ATENÇÃO_

É importante utilizarmos esse comando com **cautela**, pois alterar nomes pode afetar scripts, consultas e aplicações que dependem dos nomes antigos. É essencial garantir a compatibilidade e atualizar todas as referências correspondentes após usar o **RENAME**.



Por exemplo, podemos renomear a tabela **HistoricoEmprego** para **CargosColaboradores** utilizando o **RENAME**:

```
ALTER TABLE HistoricoEmprego RENAME TO  
CargosColaboradores;
```



UTILIZE E DOMINE O SQL!

Parabéns por explorar o Glossário de SQL! Agora que você adquiriu os fundamentos essenciais, é hora de aplicar esse conhecimento. Utilize este material como referência em seus projetos e desafios, praticando para aprimorar suas habilidades na manipulação de bancos de dados. Ao se tornar mais confiante na linguagem SQL, você estará preparado para enfrentar novos desafios.

Muito obrigado por chegar até aqui e nos vemos nos próximos cursos da formação em SQL da Alura. **Até mais!**

Avalie o curso e deixe um comentário.

Compartilhe um resumo de seus novos conhecimentos em suas redes sociais.

alura



Escola Data Science